

Author: Sujith Singh | Role: Member 3 - Storage & Delta | Weeks 1 & 2

```
[User Code to Trace] -> [Member 1: AST] -> [Member 2: Tracer (sys.settrace)]
```

```
+-----+
| MEMBER 3: STORAGE & DELTA (YOUR SUBSYSTEM!)                               |
|   [manager.py]      <-- The 'Remote Control'                             |
|       |                                                     |
|       +---> [serializer.py] <-- 'Translator' (cleans objects, catches cycles)|
|       |                                                     |
|       v                                                     |
|   [database.py]     <-- 'Filing Cabinet' (saves in strict 1,2,3 order)    |
|       |                                                     |
|       v [SQLite Database: trace.db / :memory:]                    |
+-----+
                                     |
                                     v provides events & variable history
[Member 4: Textual UI Slider]
```

"Good morning evaluators! My name is Sujith Singh, Member 3 on Team PyChronicle, responsible for the Storage & Delta Subsystem.

Standard debuggers are linear: if a bug happens on line 50, you must restart from zero. PyChronicle allows developers to scrub backward and forward through time like a video player. My responsibility is the Data Engine: saving every line executed and variable state into SQLite in strict sequence order, safely converting complex objects without crashing, and providing a clean, zero-SQL API for my teammates.

All 26 automated unit tests pass, and our benchmark handles 10,000 events at >20k/sec."

1. `python demo_phase1_phase2.py` (Visual demo showing time-travel & watch table)
2. `python -m pytest -v` (Runs all 26 automated tests -> all green PASSED)
3. `python benchmark.py` (Performance stress test: >20,000 events/second)

SECTION 2: HOW TEAMMATES BENEFIT FROM MY WORK

When evaluators ask: 'How does your work connect with the rest of the team?'

1. Member 2 - Execution Tracer (The 'Camera')

* Benefit: Member 2 does not need to learn SQL, handle database locks, or worry about serializing tricky Python objects.

* In their sys.settrace callback, they simply write ONE line:

```
storage.record_event(line_number=frame.f_lineno, state=frame.f_locals)
```

* My engine handles transactions, serialization, and disk saving invisibly.

2. Member 4 - Textual UI (The 'Screen / Video Player')

* Benefit: Member 4 does not need to parse memory dumps or raw files.

* They simply call my clean retrieval methods to power their widgets:

```
events = storage.get_events()           # Feeds the timeline slider
history = storage.get_variable_history("x") # Feeds Watch Variable panel
```

* My batch querying fetches all data in 2 queries, so their UI never freezes.

3. Member 1 - AST Rewriter (The 'Highlighter')

* Injected code flows through Member 2's tracer and is guaranteed to be stored deterministically by my sequence ordering engine.

SECTION 3: SUPER SIMPLE FILE-BY-FILE BREAKDOWN (PART 1)

1. pychronicle/storage/models.py

Real-Life Analogy: The 'Identity Card / Passport'

* What problem it solves: Prevents passing messy dictionaries with random keys.

* What it actually does: Creates clean, official blueprints for data in memory:

- ExecutionRecord: Stores the script name and start time.
- TraceEvent: Stores one line hit (e.g. Line 10, Step 1).
- VariableState: Stores one variable name and its value (e.g. x = 5).

* Week 3 Hook: Added is_delta: bool = False to support delta compression smoothly.

2. pychronicle/storage/serializer.py

Real-Life Analogy: The 'Translator & Airport Security'

* What problem it solves: SQLite only understands plain text and numbers. Complex Python objects or circular lists like a = [a] cause standard tools to crash.

* What it actually does:

- Converts numbers, strings, and lists into safe JSON text strings.
- Cycle Detection: Tracks memory IDs (id(obj)) so circular references like a=[a] are safely labeled '<CircularReference>' instead of crashing with infinite recursion.
- Safe Fallbacks: Safely converts open files, sockets, and functions to clean labels.

SECTION 4: WHAT WE ARE DOING IN FURTHER WEEKS

Week 3: Delta Compression & State Reconstruction

- * Current Situation: Right now, if a program has 10 variables, we save all 10 at every line.
- * Week 3 Objective: Build Delta Compression (cutting storage footprint by ~90%).
- * How it works:
 1. State-Diff Calculator (delta.py): Compares new state vs old state. If only variable x changed on line 20, we ONLY save variable x!
 2. State Reconstructor (reconstructor.py): Replays deltas up to step K on demand to rebuild the complete variable state for the UI.
 3. Keyframe Snapshots: Saves a full snapshot every 50 steps so historical scrubbing in the TUI is instantaneous.
- * Schema Readiness: The is_delta flag is already built into our database table!

Week 4: Packaging & Final Polish

- * Package PyChronicle as a command-line tool (e.g. pychronicle run myscript.py).
- * Assist Member 4 with connecting TUI Watch Variables panel to our history queries.

PRESENTATION QUICK-REFERENCE CHEAT SHEET

1. What 3 commands do I execute?

python demo_phase1_phase2.py	-> Live visual time-travel playback and watch table
python -m pytest -v	-> All 26 automated unit tests pass 100%
python benchmark.py	-> Performance audit: >20,000 events/second

2. Why SQLite instead of JSON files?

"SQLite gives indexed lookups on (execution_id, sequence). For 10,000 lines of code, parsing a giant JSON file freezes the UI; SQLite queries return in milliseconds."

3. Why not Python pickle?

"Pickle can execute arbitrary malicious code upon unpickling and is brittle across Python versions. Our custom JSON serializer with safe fallbacks is safe and inspectable."

4. What is Delta Compression in Week 3?

"Delta compression only saves variables that actually mutated on each line instead of the full state tree, reducing database and memory usage by 90% without schema changes."